

[开发指南](#)

LangChain 集成

 Copy page

智谱支持兼容 LangChain 框架，让您可以使用 LangChain 的强大功能来构建复杂的 AI 应用。

LangChain 是一个用于开发由语言模型驱动的应用程序的框架。智谱与 LangChain 的集成让您能够：

- 使用 LangChain 的链式调用功能
- 构建智能代理和工具调用
- 实现复杂的对话记忆管理

核心优势



框架生态

接入 LangChain 丰富的生态系统和工具链



快速开发

使用预构建的组件快速构建复杂 AI 应用



模块化设计

灵活组合不同的组件满足各种需求



社区支持

享受活跃的开源社区和丰富的资源

环境要求



Python 版本

Python 3.8 或更高版本



LangChain 版本

langchain_community 版本在 0.0.32 以上

⚠ 请确保 langchain_community 的版本在 0.0.32 以上，以获得最佳的兼容性和功能支持。

安装依赖

基础安装

```
# 安装 LangChain 和相关依赖
pip install langchain langchainhub httpx_sse

# 安装 OpenAI 兼容包
pip install langchain-openai
```

完整安装

```
# 一次性安装所有相关包
pip install langchain langchain-openai langchainhub httpx_sse

# 验证安装
python -c "import langchain; print(langchain.__version__)"
```

快速开始

获取 API Key

1. 访问 [智谱开放平台](#)

4. 复制您的 API Key 以供使用

💡 建议将 API Key 设置为环境变量： `export ZAI_API_KEY=your-api-key` 替代硬编码到代码中，以提高安全性。

基础配置

```
import os
from langchain_openai import ChatOpenAI

# 创建智谱 LLM 实例
llm = ChatOpenAI(
    temperature=0.6,
    model="glm-5.1",
    openai_api_key="your-zhipuai-api-key",
    openai_api_base="https://open.bigmodel.cn/api/paas/v4/"
)

# 或者使用环境变量
llm = ChatOpenAI(
    temperature=0.6,
    model="glm-5.1",
    openai_api_key=os.getenv("ZAI_API_KEY"),
    openai_api_base="https://open.bigmodel.cn/api/paas/v4/"
)
```

基础使用示例

简单对话



```
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage, SystemMessage
```

```
# 创建 LLM 实例
```

```
llm = ChatOpenAI(
    temperature=0.7,
    model="glm-5.1",
    openai_api_key="your-zhipuai-api-key",
    openai_api_base="https://open.bigmodel.cn/api/paas/v4/"
)
```

```
# 创建消息
```

```
messages = [
    SystemMessage(content="你是一个有用的 AI 助手"),
    HumanMessage(content="请介绍一下人工智能的发展历程")
]
```

```
# 调用模型
```

```
response = llm(messages)
print(response.content)
```

使用提示模板



```
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
```



```
# 创建 LLM
llm = ChatOpenAI(
    model="glm-5.1",
    openai_api_key="your-zhipuai-api-key",
    openai_api_base="https://open.bigmodel.cn/api/paas/v4/"
)

# 创建提示模板
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个专业的{domain}专家"),
    ("human", "请解释一下{topic}的概念和应用")
])

# 创建链
chain = prompt | llm

# 调用链
response = chain.invoke({
    "domain": "机器学习",
    "topic": "深度学习"
})

print(response.content)
```

对话记忆管理



```
from langchain_openai import ChatOpenAI
from langchain.prompts import (
    ChatPromptTemplate,
    MessagesPlaceholder,
    SystemMessagePromptTemplate,
    HumanMessagePromptTemplate,
)
from langchain.chains import LLMChain
from langchain.memory import ConversationBufferMemory

# 创建 LLM
llm = ChatOpenAI(
    temperature=0.6,
    model="glm-5.1",
    openai_api_key="your-zhipuai-api-key",
    openai_api_base="https://open.bigmodel.cn/api/paas/v4/"
)

# 创建提示模板
prompt = ChatPromptTemplate(
    messages=[
        SystemMessagePromptTemplate.from_template(
            "You are a nice chatbot having a conversation with a human."
        ),
        MessagesPlaceholder(variable_name="chat_history"),
        HumanMessagePromptTemplate.from_template("{question}")
    ]
)

# 创建记忆
memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)

# 创建对话链
conversation = LLMChain(
    llm=llm,
    prompt=prompt,
    verbose=True,
    memory=memory
)
```

进行对话

BigModel

response1 = conversation.invoke({"question": "tell me a joke"})

print("AI:", response1['text'])

>

response2 = conversation.invoke({"question": "tell me another one"})

print("AI:", response2['text'])

高级功能

智能代理 (Agent)

```
from langchain import hub

from langchain.agents import AgentExecutor, create_react_agent
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_openai import ChatOpenAI

# 设置搜索工具 API 密钥
os.environ["TAVILY_API_KEY"] = "your-tavily-api-key"

# 创建 LLM
llm = ChatOpenAI(
    model="glm-5.1",
    openai_api_key="your-zhipuai-api-key",
    openai_api_base="https://open.bigmodel.cn/api/paas/v4/"
)

# 创建工具
tools = [TavilySearchResults(max_results=2)]

# 获取提示模板
prompt = hub.pull("hwchase17/react")

# 创建代理
agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# 执行任务
result = agent_executor.invoke({"input": "what is LangChain?"})
print(result['output'])
```

自定义工具



```
from langchain.tools import tool
from langchain.agents import AgentExecutor, create_react_agent

from langchain import hub
from langchain_openai import ChatOpenAI


@tool
def get_weather(city: str) -> str:
    """获取指定城市的天气信息"""
    # 这里应该调用真实的天气 API
    # 示例返回
    return f"{city} 的天气: 晴天, 温度 25°C, 湿度 60%"


@tool
def get_stock_price(symbol):
    """获取股票价格"""
    # 模拟股票 API 调用
    return {
        "symbol": symbol,
        "price": 150.25,
        "change": "+2.5%"
    }


# 创建 LLM
llm = ChatOpenAI(
    model="glm-5.1",
    openai_api_key="your-zhipuai-api-key",
    openai_api_base="https://open.bigmodel.cn/api/paas/v4/",
)


# 工具列表
tools = [get_weather, get_stock_price]


# 创建代理
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True, max_iterati


# 使用代理
```

流式输出

```
from langchain_openai import ChatOpenAI
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
from langchain.schema import HumanMessage

# 创建带流式输出的 LLM
llm = ChatOpenAI(
    model="glm-5.1",
    openai_api_key="your-zhipuai-api-key",
    openai_api_base="https://open.bigmodel.cn/api/paas/v4/",
    streaming=True,
    callbacks=[StreamingStdOutCallbackHandler()]
)

# 发送消息（输出会实时流式显示）
response = llm([HumanMessage(content="写一首关于春天的诗")])
```

实践建议

性能优化

- 启用 LangChain 缓存机制
- 使用批量处理减少 API 调用
- 合理设置 max_tokens 限制
- 使用异步处理提高并发性能

错误处理

- 实施重试机制和指数退避
- 设置合理的超时时间
- 记录详细的错误日志
- 提供降级方案

内存管理

安全性

- 使用环境变量存储 API 密钥



使用 BigModel

- 使用 ConversationBufferWindowMemory 限制历史长度
- 定期清理不必要的对话历史
- 监控内存使用情况
- 实施对话摘要机制

- 实施输入验证和过滤
- 监控 API 使用量和成本
- 定期轮换 API 密钥



更多资源



智谱 API 文档

查看智谱完整的 API 文档



LangChain 官方文档

查看 LangChain 官方文档和教程

❗ LangChain 是一个快速发展的框架，建议定期更新到最新版本以获得最佳功能和性能。同时，智谱会持续优化与 LangChain 的集成，确保最佳的兼容性和用户体验。

◀ OpenAI API 兼容

迁移至 GLM-5.1 ▶